THE NATIONAL UNIVERSITY OF LESOTHO

Department of Mathematics and Computer Science

CS4433: SOFTWARE ENGINEERING

FINAL PROJECT REPORT

Moderated Learning Management System (LMS)

Bakoena Thamae | 202321319 | BSc Information Systems

Main report with hyperlinks to external project artefacts

Abstract

This report summarises the planning, requirements engineering, system design, implementation, verification, quality assurance, deployment and maintenance of a Moderated Learning Management System for a secondary school. The application supports Administrator, Lecturer, Head of Department (HoD) and Student roles. It is designed as an offline C++17 console application using SQLite for runtime persistence, CMake for builds and OpenSSL for salted password hashing. The project follows the eight phases in the CS4433 individual assignment. Supporting reports, editable diagrams, SQL, test evidence, manuals and release documents are stored as separate files in the accompanying GitHub repository and linked from this report rather than embedded as large images or appendices.

1. Project context and objectives

The school needs a local system to manage departments, courses, lecturer assignments, student enrolments, assignments, submissions, feedback, moderation and controlled release of marks. The four roles have different responsibilities and access scopes. Students must see only their own released results; lecturers work only with assigned courses; HoDs moderate and release marks for their departments; administrators maintain the academic structure and user accounts.

The project must operate without an internet connection during normal use, protect passwords, retain mark-change audit history and remain usable with approximately 800 students. Online payments, video conferencing, automated grading, plagiarism detection and web/mobile interfaces are outside the agreed scope.

2. Project repository

The complete project repository contains the phase reports, implementation, editable diagrams and their previews, test code, evidence logs, CI workflow, deployment materials, maintenance artefacts and recording folder. The repository README is the navigation index; the linked artefacts are maintained independently so that source files and diagrams remain editable.

Open the Moderated LMS GitHub repository

3. Phases 1–3: planning, requirements and design

3.1 Phase 1 — Project planning and management

The project plan assesses technical, operational, economic, legal and schedule feasibility and recommends proceeding subject to scope control, appropriate security, performance testing and user support. A work breakdown structure and seven-week Waterfall schedule define dependencies and phase gates. The critical chain is planning → requirements → design → implementation → verification → deployment → maintenance. The risk register covers technical, schedule and people risks and is intended to be updated at each gate.

Phase 1 Project Management Plan | Seven-week Gantt Chart

3.2 Phase 2 — Requirements engineering

The requirements specification identifies stakeholders, resolves conflicts and defines functional and non-functional requirements with verification criteria. Must requirements include authentication, role-based access, administration, academic structure, assignment creation, individual extensions, submission/resubmission, late penalties, marking, moderation, release, result visibility and audit logging. Non-functional requirements cover performance, offline operation, password protection, access control, data integrity, auditability, usability, C++17 compatibility and recoverability. The late penalty is 10% of the assignment maximum per late day for up to three days; the Phase 2 document records calendar days as an assumption pending confirmation.

Phase 2 Requirements / SRS | Editable Use Case Diagram | Use Case PNG

3.3 Phase 3 — System design

The design translates requirements into an offline application architecture, class and behavioural models, database entities and traceability. The diagram set includes architecture, class, submission state machine, activity/swimlane, submission sequence and mark moderation/release sequence diagrams. The MySQL 8 schema is a Phase 3 design artefact and supports the logical LMS ERD. Phase 4 implementation uses SQLite instead, as required by the assignment; this distinction is maintained in the repository to avoid treating the MySQL design database as the live runtime database.

Revised Phase 3 System Design Report (DOCX) | PDF

Phase 3 editable diagrams and previews | Logical LMS ERD | MySQL design DDL

Tool-evidence qualification: the supplied MySQL Workbench screenshot is retained unchanged but displays a separate timetable schema. It is not presented as the Moderated LMS EER diagram. A genuine Workbench EER export should be added if the supervisor requires that specific tool evidence.

4. Phases 4–5: implementation and verification

4.1 Phase 4 — Implementation

The application is written in C++17. An abstract User class and role-specific subclasses implement Administrator, Lecturer, HoD and Student behaviour using inheritance and polymorphism. SQLite is used for local persistent storage through a data-access wrapper; user-supplied values are passed through prepared statements. OpenSSL PBKDF2-HMAC-SHA256 with per-password random salt is used to store password hashes. Marking, moderation, release and corresponding audit entries are coordinated using database transactions. The current source includes CR1 class-wide extension support; the original Phase 4 baseline is archived under releases/v1.0.

C++ application source | Business rules module | SQLite schema | CMake build configuration

4.2 Phase 5 — Verification and validation

The V-Model test strategy links unit tests to business rules, integration tests to repositories and services, system tests to end-to-end use cases, and acceptance tests to stakeholder requirements. Equivalence partitioning and boundary-value analysis are used for date validity, late-day calculations, deadlines, resubmissions, mark ranges and extension policies. Catch2 automated test code and a manual assertion harness are included. System/acceptance matrices and a defect log are supplied separately.

Executed check (Linux development container, 1 Nov 2026)	Result	Qualification
C++17 production configure/build and database initialization	PASS	Development container only
Ten-table runtime/schema parity and foreign-key integrity	PASS	SQLite schema comparison
Legacy database migration adds extension_scope and defaults old rows to INDIVIDUAL	PASS	Existing data preserved
Business-rule smoke assertions and CR1 workflow across restart	PASS	Scope described in evidence files
Manual-harness business-rule coverage	84.18% lines; 78.91% branches executed	Preliminary; not Catch2/CI coverage
Catch2/CTest, GitHub Actions coverage and Windows acceptance	PENDING	Must run and attach evidence

Test strategy | Catch2 tests | Execution results | Coverage/build evidence | Defect log

The 84.18% line execution figure is from the manual business-rule assertion harness only. It must not be described as the final Catch2/CI coverage result. Catch2 could not be downloaded in the earlier build environment because GitHub could not be resolved, and target Windows acceptance tests have not yet run.

5. Phases 6–7: quality assurance and deployment

5.1 Phase 6 — Configuration management and QA

The repository uses Git with a feature-branch and pull-request workflow, meaningful commit messages and intended v1.0/v1.1 release tags. A GitHub Actions workflow is configured to build the C++ project, run Catch2/CTest, produce business-logic coverage and run static analysis on pushes and pull requests. The workflow file is present, but the live successful run, commit SHA and CI artifact still need to be captured after the complete repository is pushed.

The recorded Clang Static Analyzer run produced no diagnostics for the analysed C++ source snapshot. Code metrics and a self-review record are supplied as supporting documents. clang-tidy and more detailed complexity measurements remain to be run in a network-enabled or local environment with the required tools; reviewer initials, review date and submitted commit SHA must be filled by the student after reviewing the exact submission commit.

GitHub Actions workflow | Configuration and QA | Metrics and static analysis | Code review record

5.2 Phase 7 — Deployment

The deployment model targets a school workstation or local network environment. The package contains a UML deployment diagram with editable PlantUML and DOT sources and PNG/SVG exports, a technician installation guide, a four-role user manual, v1.0 release notes and deployment-verification checklist. The guide documents required C++ build tools, CMake, SQLite/OpenSSL dependencies, database creation, first-run administrator account and local execution. The initial password must be changed before real use. Final installation and usability acceptance on the designated school computer remain pending.

Deployment diagram (PNG) | Installation guide | Four-role user manual | v1.0 release notes

5.3 Release evidence

A v1.0 source snapshot is preserved under releases/v1.0/source. The repository does not claim that GitHub tags have already been created. Apply the v1.0 tag after acceptance of the initial implementation and deployment; apply v1.1 only after CR1 regression and CI checks have passed.

6. Phase 8 — Maintenance and CR1

CR1 adds lecturer-authorised class-wide deadline extensions and shows each Student the effective deadline. This is perfective maintenance of the existing extension workflow. The implementation adds INDIVIDUAL and CLASS scope values, preserves individual exceptions, applies class updates transactionally to current enrolments, and displays the effective deadline and extension label. Existing databases receive an additive migration; earlier rows default to INDIVIDUAL.

Affected artefacts include FR-08/business rules, UML use-case/activity/sequence models, extension service, SQLite migration, lecturer menu, student assignment query, tests, manual and release notes. Local regression checks passed for class roster updates, individual exceptions, lecturer ownership, invalid/empty rosters, transaction rollback and display after restart. Full marking/moderation/audit regression remains pending.

CR1 impact analysis | Updated design | Regression test matrix | Maintenance verification | v1.1 release notes

Before v1.1 acceptance, the supervisor must confirm that individual extensions remain exceptions (current behaviour) and that later-enrolled students do not automatically inherit an earlier class grant (current behaviour).

7. Professional, legal and ethical considerations

The LMS processes learner identities, enrolments, submissions and marks, so the Lesotho Data Protection Act 2011 (Act No. 5 of 2012) is relevant. Controls include limiting data to assessment purposes, role/course/department access, salted password hashes, mark-change audits and controlled backups/files. The school must confirm retention periods, learner access/correction, breach response and controller responsibilities before deployment.

Assessment records must remain confidential and accurate: students see only released results, lecturers act only in assigned courses and HoDs moderate within their departments. Late rules must be transparent and extensions justified. Test keyboard usability, clear messages and readable output with representative users. Use synthetic accounts in screenshots/video; never publish identifiable learner data in the public repository.

8. Conclusion

The repository brings together Phases 1–8, the C++/SQLite implementation, CR1 change, editable diagrams and supporting evidence. Final completion still requires successful Catch2/CI, Windows acceptance, performance/offline/usability/restore checks, reviewer details, supervisor policy approval and genuine v1.0/v1.1 tags.

References

International Organization for Standardization, IEC and IEEE (2018) ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO.

Lesotho (2012) Data Protection Act 2011 (Act No. 5 of 2012). Government Gazette, Act 5 of 2012. Available at: https://lesotholii.org/akn/ls/act/2012/5/eng%402012-02-22 (accessed 2 November 2026).

National University of Lesotho, Department of Mathematics and Computer Science (2026) CS4433: Software Engineering — Individual Assignment: Moderated Learning Management System. Maseru: National University of Lesotho.

Object Management Group (2017) OMG Unified Modeling Language (OMG UML), Version 2.5.1. Available at: https://www.omg.org/spec/UML/2.5.1 (accessed 2 November 2026).

Sommerville, I. (2016) Software Engineering. 10th edn. Boston: Pearson.